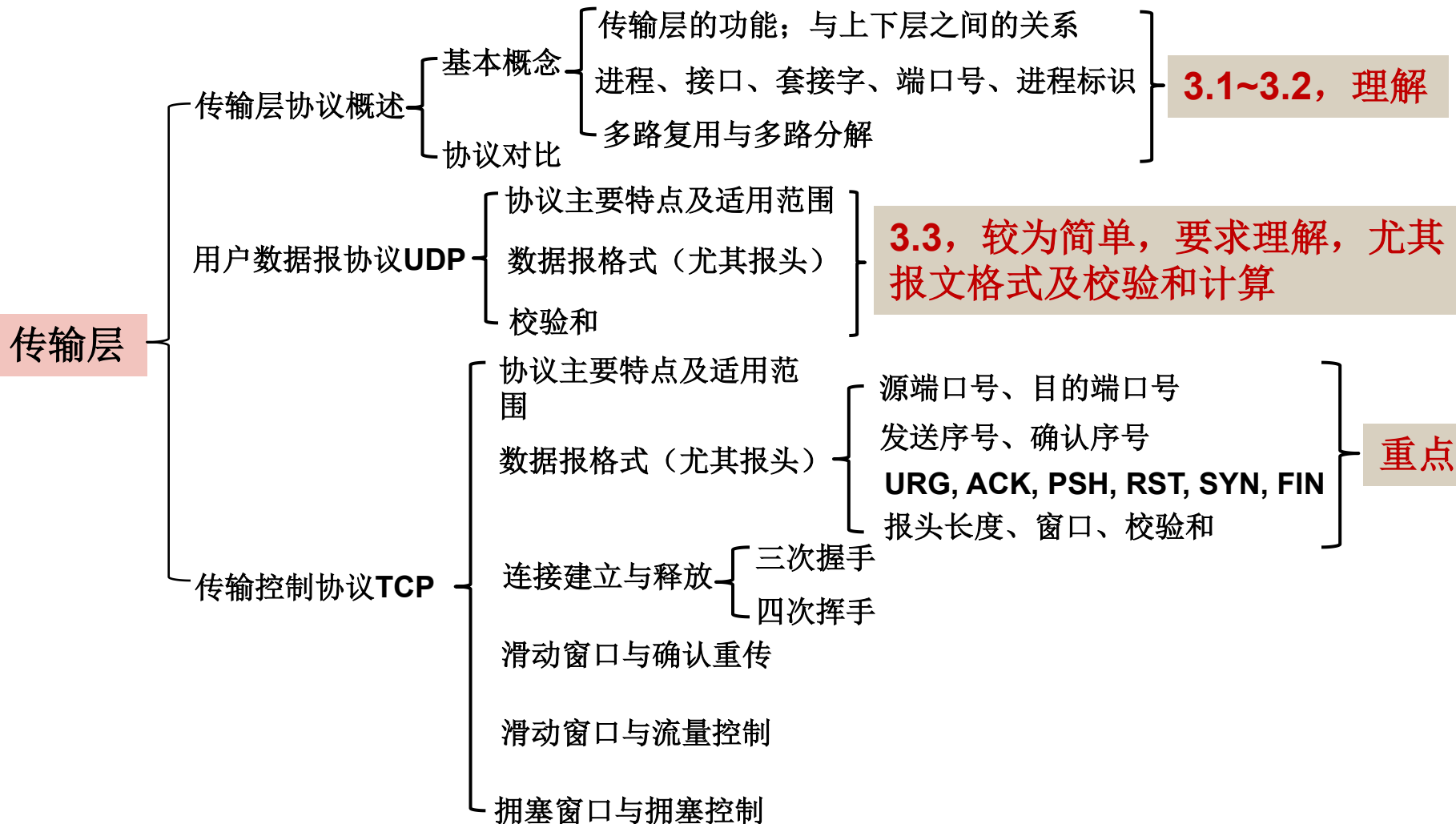


第三章 传输层

计算机网络



本章概览



3.1 传输层的基本概念

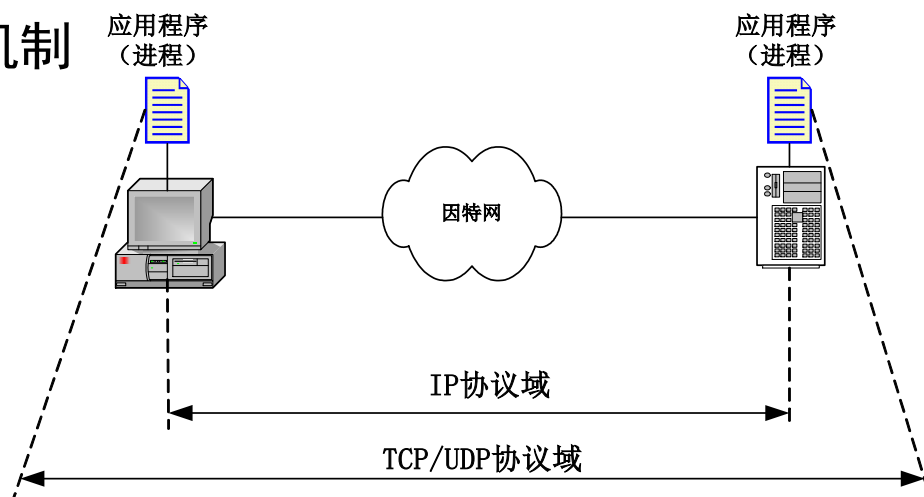
3.1.1 传输层的基本功能

- 根本目的：在网络层提供的数据通信服务基础上，实现主机进程间可靠服务，实现“端到端”服务（主机-主机 → 端-端）

传输层的不同协议可靠的内涵有所不同，如UDP的可靠性弱于TCP的可靠性。

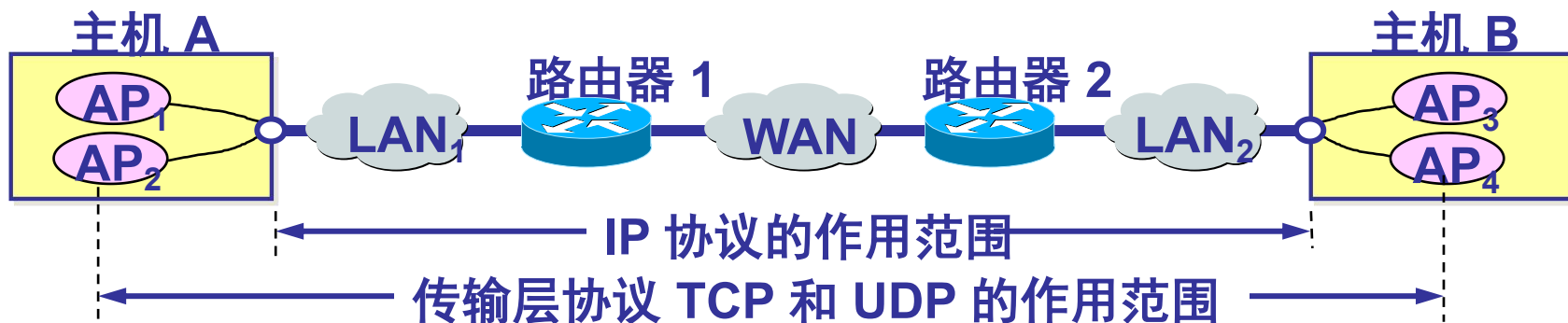
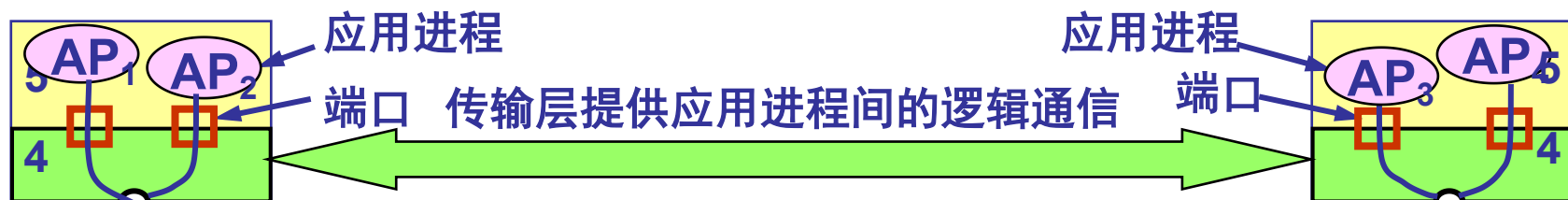
两大功能：

- 加强、弥补网络层提供的网络服务
- 进一步提供进程间“可靠”通信机制
- 传输层协议软件属于OS内核软件
- 而非用户软件



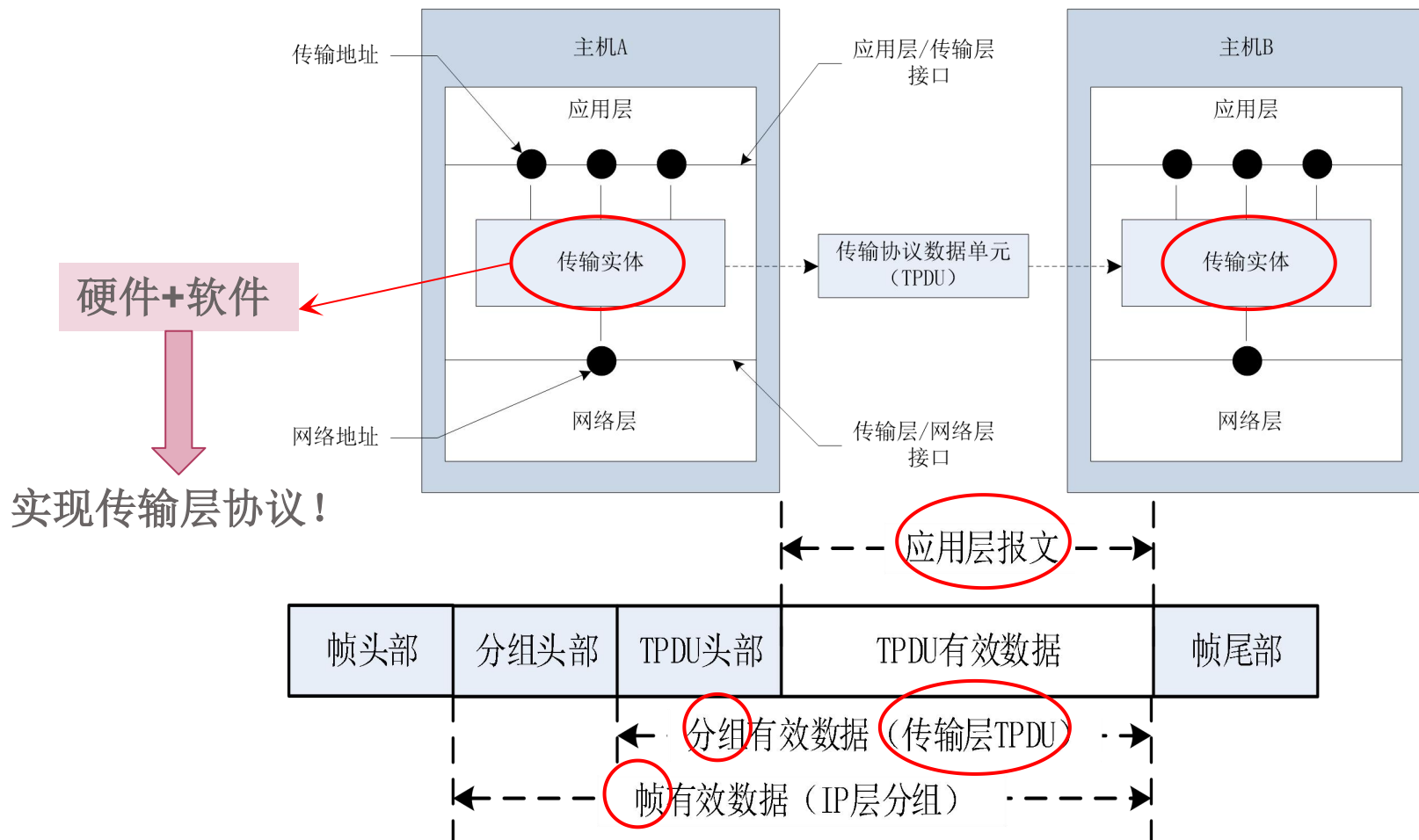
3.1 传输层的基本概念

3.1.2 传输层与应用层和网络层的关系



3.1 传输层的基本概念

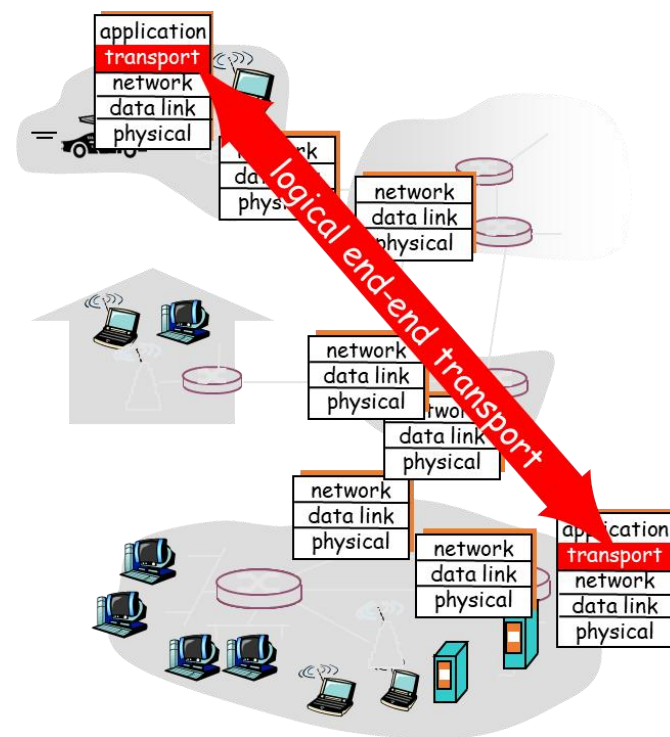
3.1.2 传输层与应用层和网络层的关系



3.1 传输层的基本概念

■ 传输层提供什么服务？

- 因特网的网络层提供“**尽力而为**”的服务：
 - 网络层尽最大努力在终端间交付分组，但不提供任何承诺
 - 具体来说，不保证交付，不保证按序交付，不保证数据完整，不保证延迟，不保证带宽等
- 传输层的**有所为、有所不为**：
 - 传输层可以通过差错恢复、重排序等手段提供可靠、按序的交付服务
 - 但传输层无法提供延迟保证、带宽保证等服务



3.1 传输层的基本概念

■ 互联传输层提供的服务

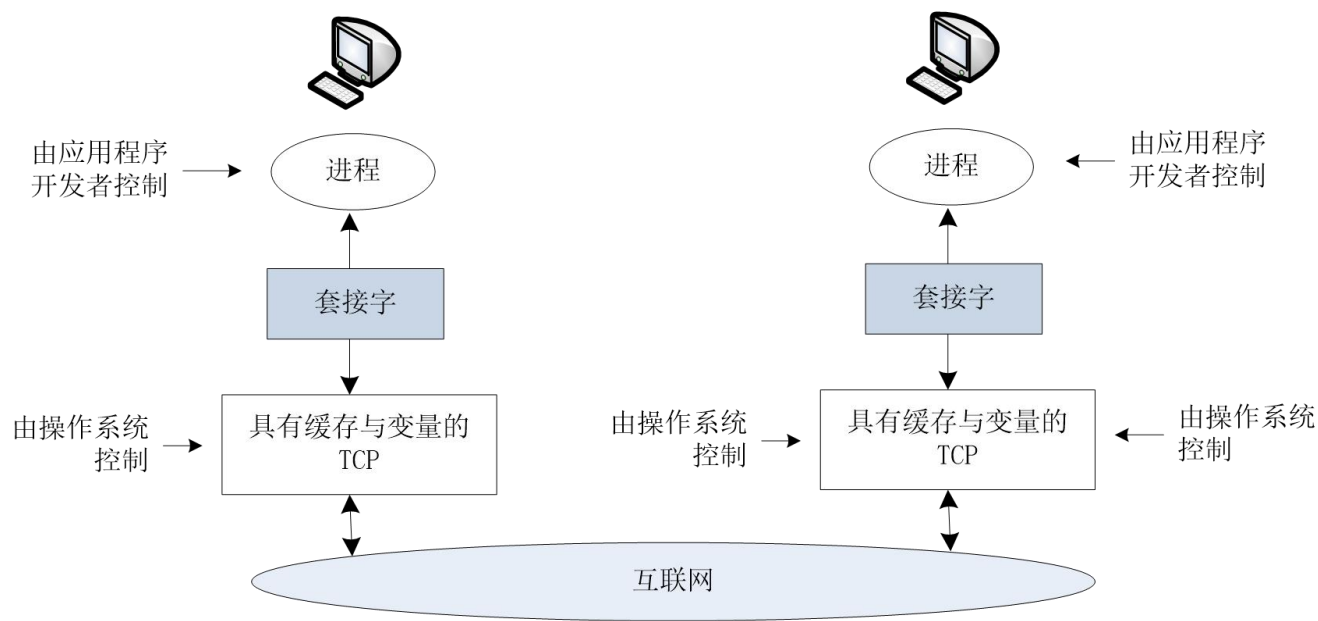
- 最低限度的传输服务：
 - 将终端-终端的数据交付扩展到进程-进程的数据交付
 - 报文检错
- 增强服务：
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 因特网传输层通过UDP协议和TCP协议，向应用层提供两种不同的传输服务：
 - UDP协议：仅提供最低限度的传输服务
 - TCP协议：提供基础服务和增强服务



3.1 传输层的基本概念

3.1.3 应用进程、传输层接口与套接字

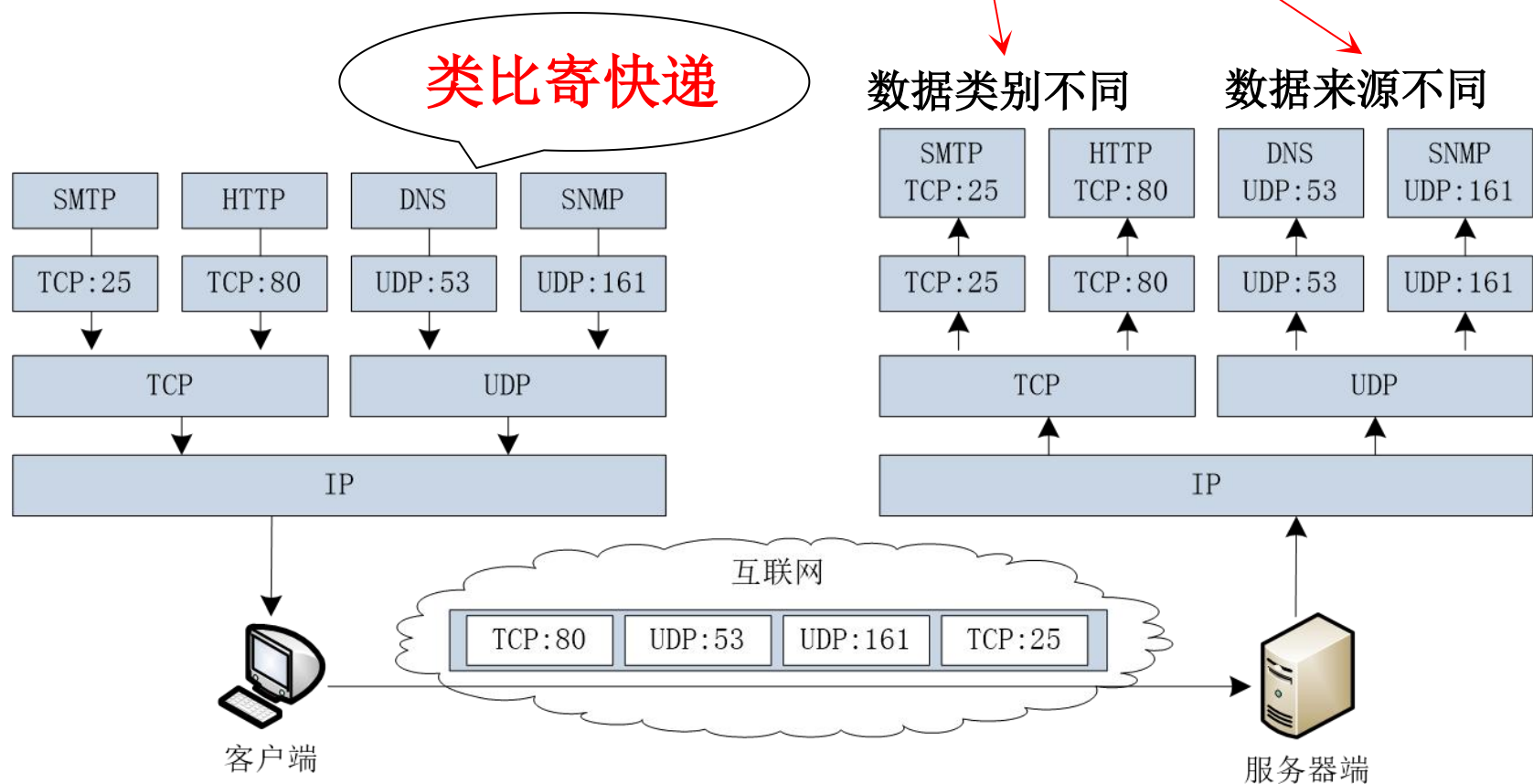
- 传输层协议在本地主机操作系统控制下，为应用程序提供确定的服务
- 网络层解决IP地址，传输层解决进程标识
- 套接字(Socket)：建立网络应用程序的可编程接口（应用编程接口API）



3.1 传输层的基本概念

3.1.4 传输层的多路复用与多路分解

- 运行TCP/IP协议主机可能同时运行不同应用层协议和应用程序



3.1 传输层的基本概念

发送端多路复用:

传输层从多个套接字收集数据,
交给网络层发送
(加报头封装, 后续多路分解)

接收端多路分解:

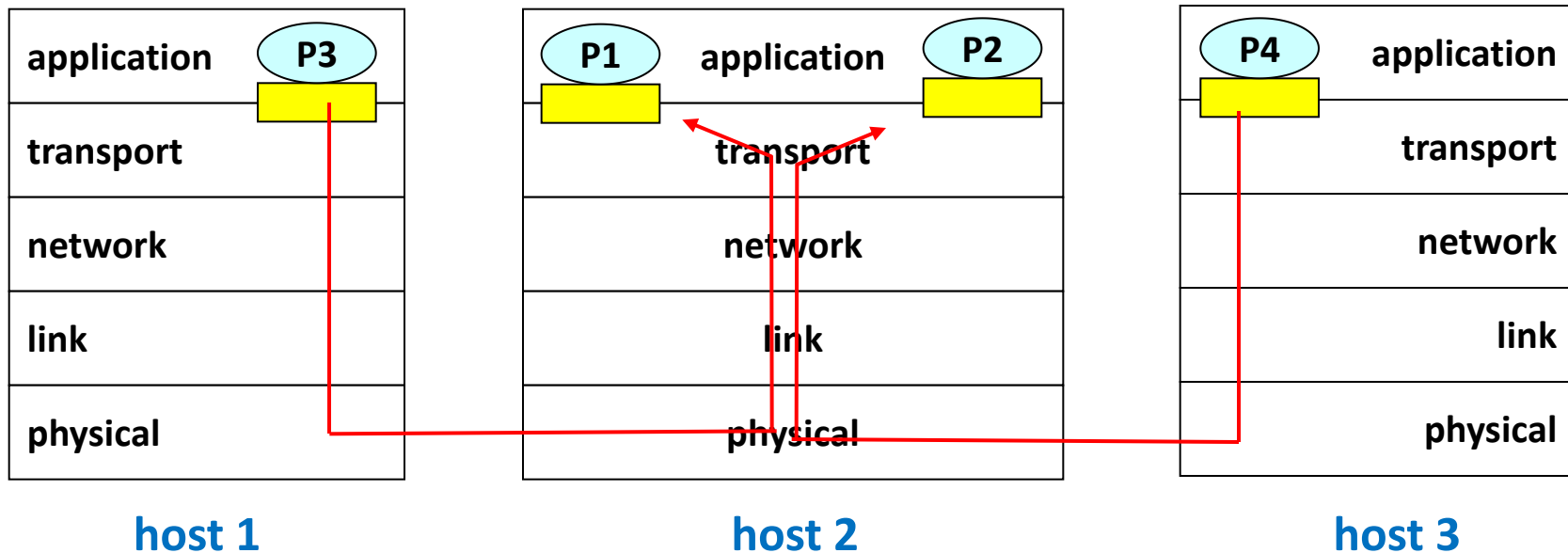
传输层将从网络层收到的数据,
交付给正确的套接字



= socket

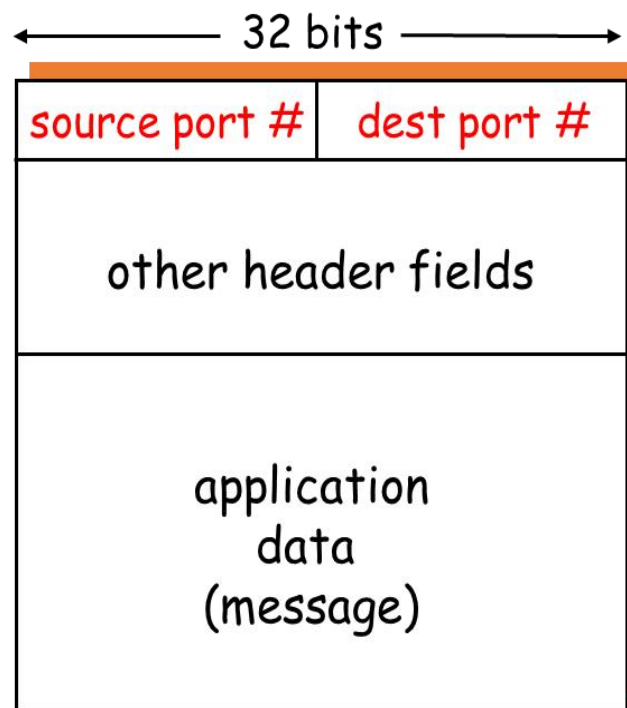


= process



3.1 传输层的基本概念

- 3.1.4 传输层的多路复用与多路分解
- 端口号是套接字标识的一部分：
 - 每个套接字在本地关联一个端口
 - 端口号是一个16比特的数
 - Q:为什么不是进程号?
- 报文段中有两个字段携带端口号
 - 源端口号：与发送进程关联的本地端口号
 - 目的端口号：与接收进程关联的本地端口号

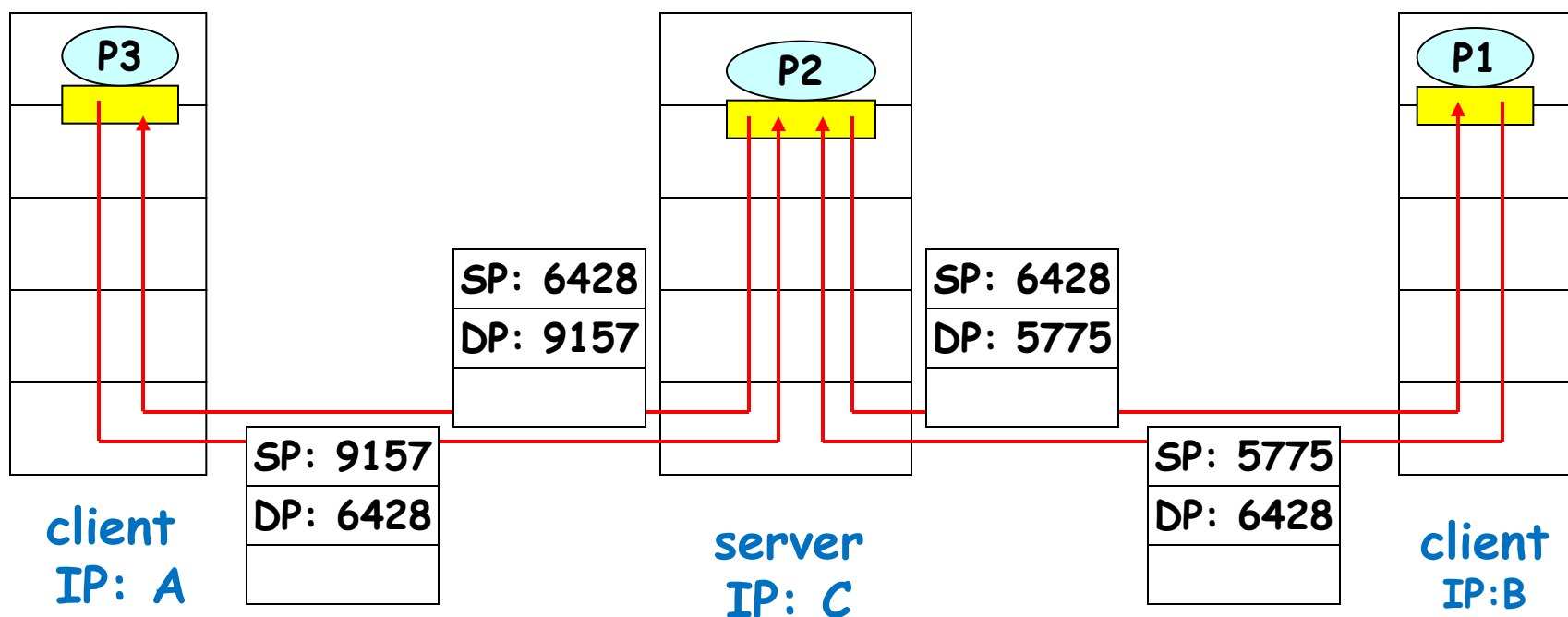


TCP/UDP报文段格式

3.1 传输层的基本概念

3.1.4 无连接的多路复用与多路分解

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```



3.1 传输层的基本概念

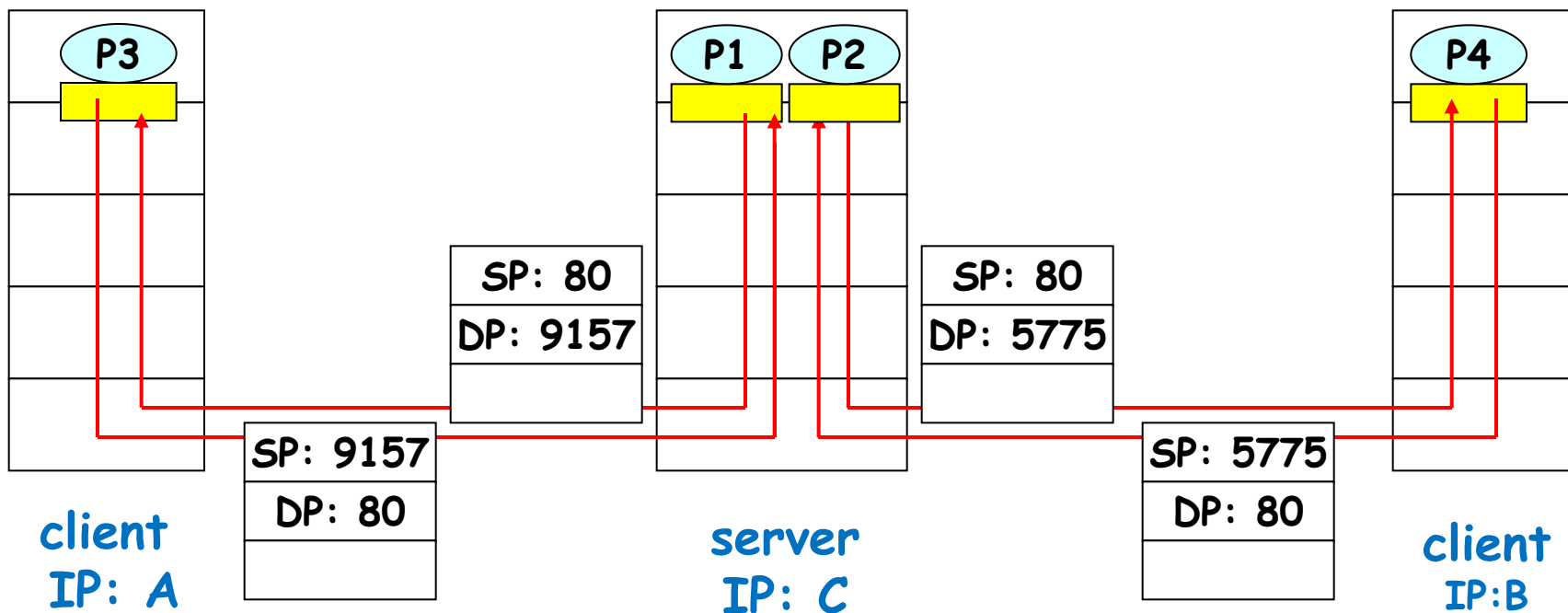
■ 3.1.4 面向连接的多路复用与多路分解

- UDP套接字
 - 使用<IP地址, 端口号>二元组标识UDP套接字
 - 服务器使用一个套接字服务所有客户
- TCP套接字
 - 使用<源IP地址, 目的IP地址, 源端口号, 目的端口号>四元组标识连接套接字
 - 服务器使用一个监听套接字和多个连接套接字服务多个客户, 每个连接套接字服务一个客户
- SOCKET socket(int af, int type, int protocol)
 - TCP: socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)
 - UDP: socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)



3.1 传输层的基本概念

3.1.4 面向连接的多路复用与多路分解



3.1 传输层的基本概念

3.1.5 网络环境中的应用进程标识

• A. 网络环境中进程通信的标识

- **进程通信标识五元组**：协议、本地地址、本地端口、远程地址、远程端口号，也叫**全相关**
- **三元组（前三）**：**半相关**

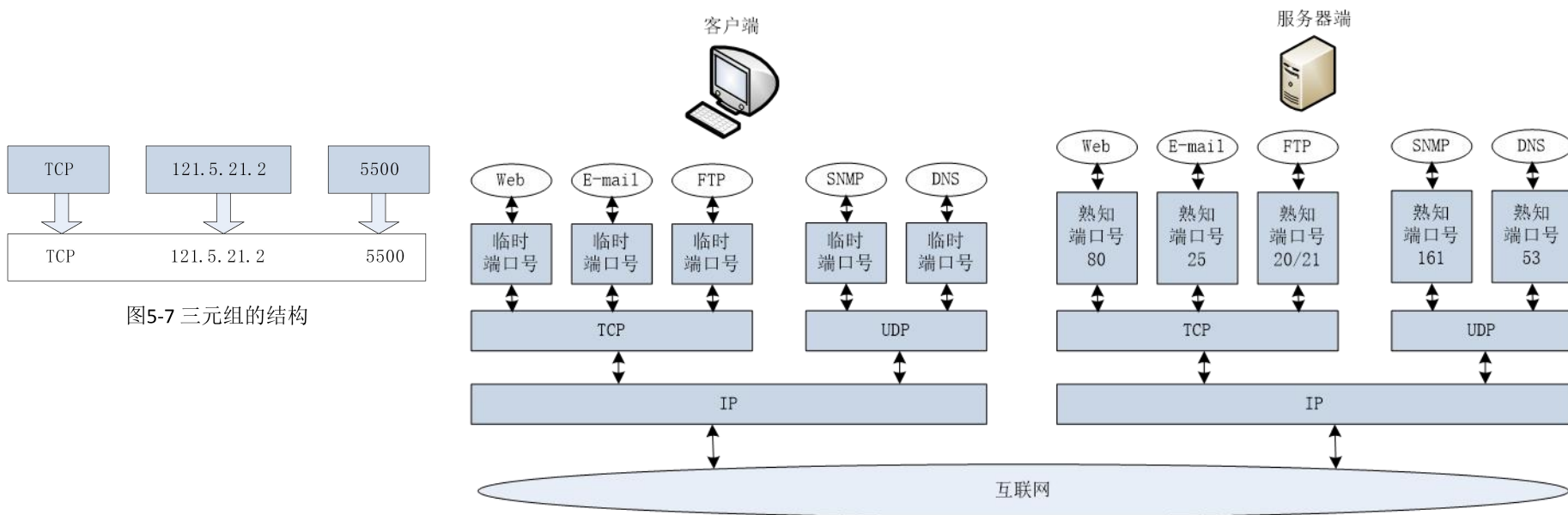


图5-7 三元组的结构

3.1 传输层的基本概念

3.1.5 网络环境中的应用进程标识

- **进程通信**标识五元组：协议、本地地址、本地端口、远程地址、远程端口号

```
root@kali:~# netstat -an
```

激活Internet连接（服务器和已建立连接的）

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	1	192.168.0.7:54470	172.217.6.127:443	SYN_SENT
tcp	0	0	192.168.0.7:39789	182.118.18.120:443	TIME_WAIT
tcp	0	1	192.168.0.7:43537	59.110.185.114:80	LAST_ACK
tcp	0	196	192.168.0.7:41847	172.217.163.238:443	FIN_WAIT1
udp	0	0	127.0.1.1:53	0.0.0.0:*	
udp	0	0	0.0.0.0:68	0.0.0.0:*	
udp	0	0	0.0.0.0:4665	0.0.0.0:*	
udp	0	0	0.0.0.0:4672	0.0.0.0:*	
udp	0	0	0.0.0.0:631	0.0.0.0:*	
udp	0	0	0.0.0.0:5353	0.0.0.0:*	

每一行表示一个传输层通信连接，如第一行表示一个通信连接
(协议=tcp, 本地地址=192.168.0.7, 本地端口=54470, 远程地址=172.217.6.127, 远程端口=443)

3.1 传输层的基本概念

3.1.4 网络环境中的应用进程标识

B. 端口号的分配方法

- 熟知端口号：给每种服务器分配的确定的全局端口号，也叫公认端口号，范围在0~1023，统一分配与控制
- 临时端口号：客户端程序使用的临时端口号，由客户端上TCP/IP软件随机选取，范围在49152~65535
- 注册端口号：在IANA（国际因特网地址分配委员会）注册的端口号，没有明确的定义服务对象，不同程序可根据实际需要自己定义，范围在1024~49151



<https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml>



3.1 传输层的基本概念

3.1.4 网络环境中的应用进程标识

表5-1 UDP常用的熟知端口号

端口号	服务进程	说 明
53	Domain	域名服务
67/68	DHCP	动态主机配置协议
69	TFTP	简单文件传输协议
111	RPC	远程过程调用
123	NTP	网络时间协议
161/162	SNMP	简单网络管理协议
520	RIP	路由信息协议

表5-2 TCP常用的熟知端口号

端口号	服务进程	说 明
20	FTP	文件传输（数据连接）
21	FTP	文件传输（控制连接）
23	TELNET	网络虚拟终端协议
25	SMTP	简单邮件传输协议
80	HTTP	超文本传输协议
119	NNTP	网络新闻传输协议
179	BGP	边界路由协议



3.2 传输层协议的特点及对比

表5-3 TCP与UDP协议比较

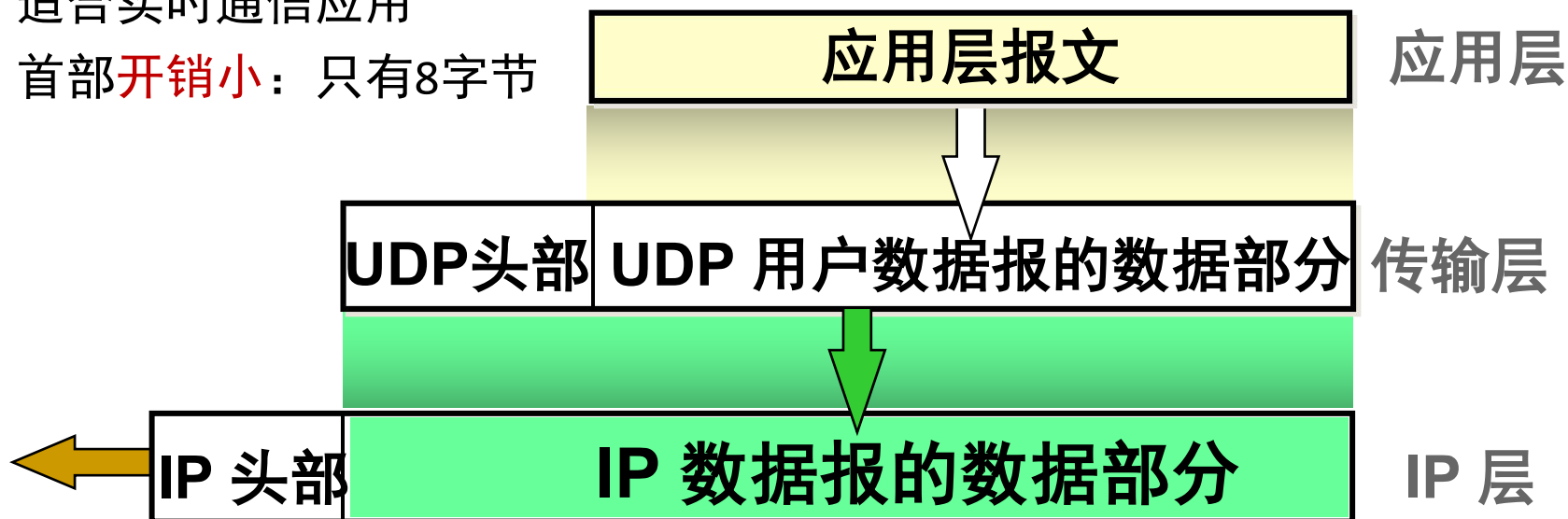
特征/描述	TCP	UDP
一般描述	允许应用程序 可靠 地发送数据， 功能齐全	简单、高速 ， 只 负责将应用层与网络层衔接起来
面向连接与无连接	面向连接 ，在TPDU传输之前需要建立TCP连接	无连接 ，在TPDU传输之前不需要建立UDP连接
与应用层的数据接口	基于字节流 ，应用层不需要规定特点的数据格式	基于报文 ，应用层需要将数据分成包来传送
可靠性与确认	可靠 报文传输，对所有的数据均要确认	不可靠 ，不需要对传输的数据确认，尽力而为地交付
重传	自动重传 丢失的数据	不负责检查 是否丢失数据和重传
开销	低 ，但高于UDP	很低
传输速率	高 ，但低于UDP	很高
适用的数据量	从少量到几个GB的数据	从少量到几百个字节的数据
适用的应用类型	对数据传输可靠性要求较高的应用，例如文件与报文传输	发送数量比较少，对数据传输可靠性要求低的应用，例如IP电话、视频会议、多播与广播



3.3 用户数据报协议UDP

3.3.1 UDP协议的主要特点

- 无连接、不可靠：开销低、尽力而为
- 面向报文的传输层协议：UDP 对应用层传递下来的报文，既不合并，也不拆分，而是保留这些报文的边界。UDP 层一次向对方交付一个完整的报文
- 没有拥塞控制：网络出现的拥塞时，UDP不会使源主机的发送速率降低，适合实时通信应用
- 首部开销小：只有8字节



3.3 用户数据报协议UDP

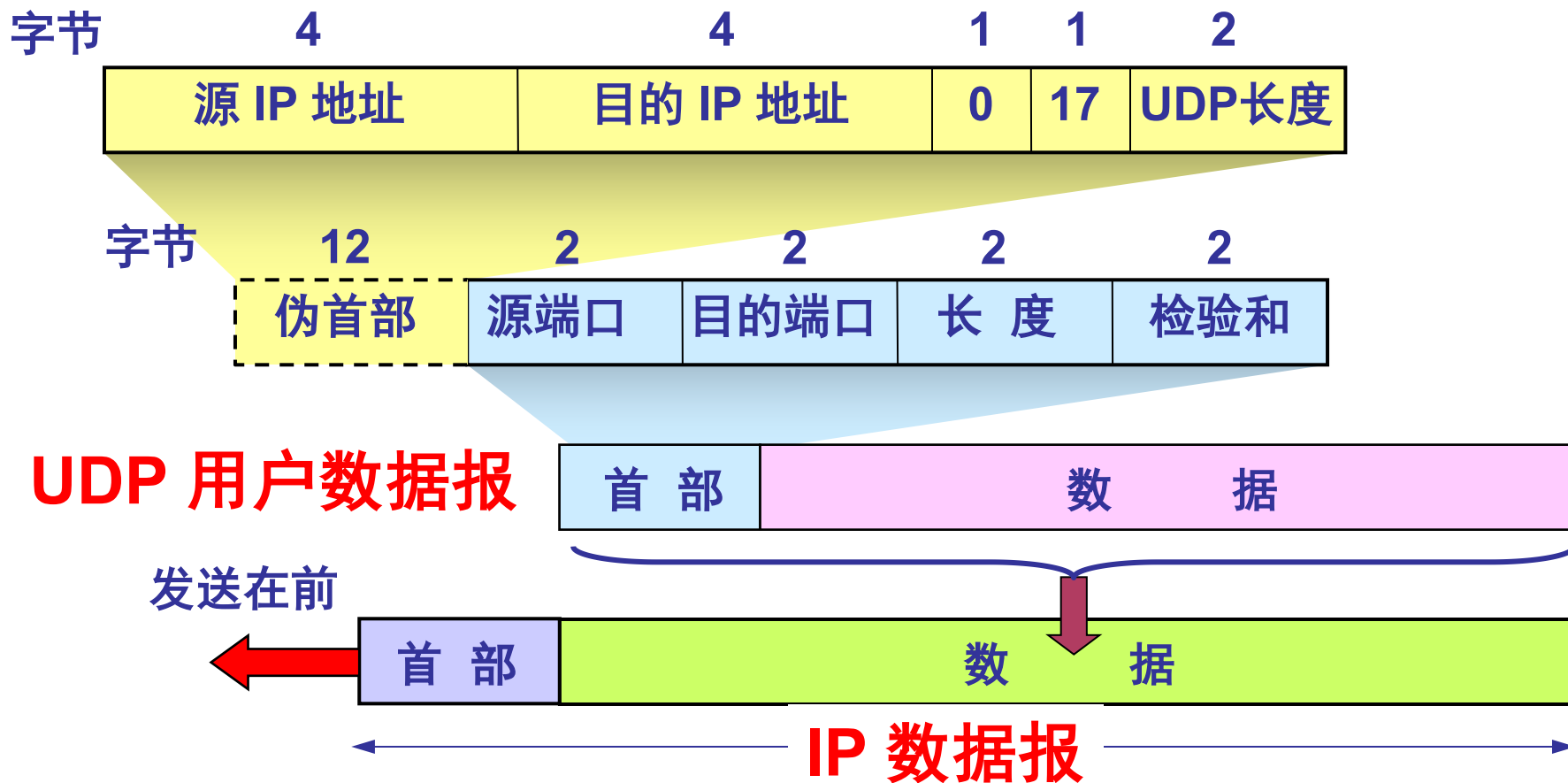
3.3.1 UDP协议的适用范围

- 对性能的要求高于对数据完整性的要求
 - 视频播放实时交付的要求高于对数据交付可靠性要求（可丢失个别数据包）
- 需要“简短快捷”的数据交换
 - 简单的请求与应答报文交互
- 需要多播和广播的应用
 - 源主机以恒定速率发送报文，拥塞发生时允许丢弃部分报文



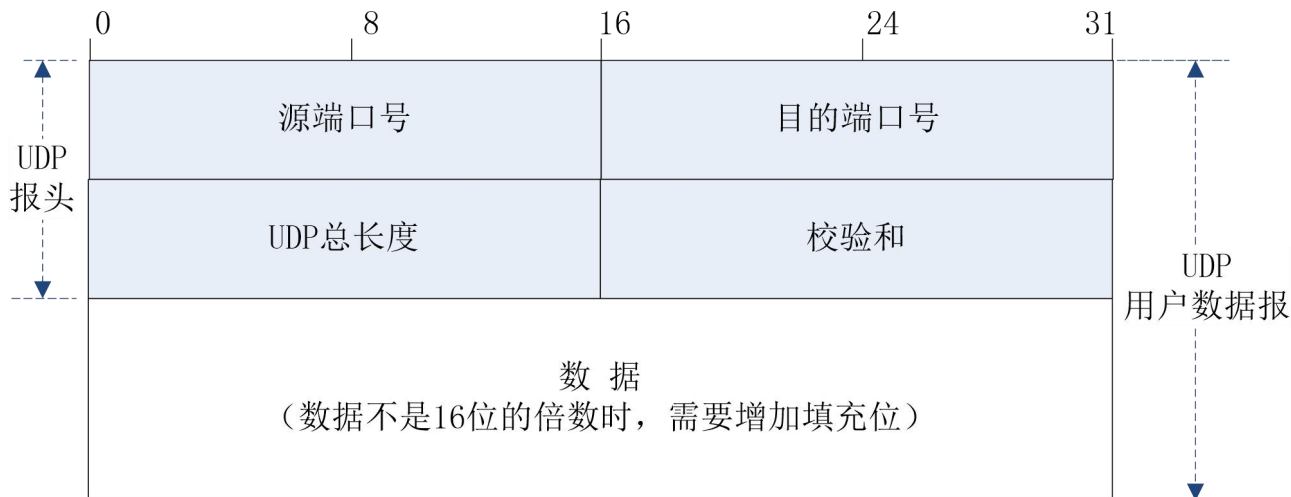
3.3 用户数据报协议UDP

3.3.2 UDP数据报格式



3.3 用户数据报协议UDP

3.3.2 UDP数据报格式



- 端口号：包括源端口号和目的端口号，分别表示发送方和接收方的进程端口号，各为2字节
- UDP总长度：包括报头在内的用户数据报的总长度，**2字节的倍数**
- **校验和**：用于检查整个数据报（含报头）是否传输出错，可选，若无，填0

3.3 用户数据报协议UDP

3.3.2 UDP数据 报格式

271	4.573484424	172.31.72.5	172.31.72.255	UDP	305 54915 → 54915 Len=26
272	4.591182803	66.65.44.189	172.31.72.133	TCP	1958 21626 → 39926 [PSH,
273	4.591234057	172.31.72.133	66.65.44.189	TCP	54 39926 → 21626 [ACK]
274	4.595506978	172.31.72.133	95.250.253.103	UDP	234 4672 → 4672 Len=192
275	4.595538361	172.31.72.133	79.151.145.66	UDP	234 4672 → 4672 Len=192
276	4.595559954	172.31.72.133	183.45.76.172	UDP	93 4672 → 55937 Len=51
277	4.595569582	172.31.72.133	27.224.145.55	UDP	234 4672 → 58399 Len=192
278	4.595617174	172.31.72.133	79.16.30.74	UDP	93 4672 → 4663 Len=51

```
.... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 220
    Identification: 0x4644 (17988)
  ▶ Flags: 0x4000, Don't fragment
    Time to live: 64
    Protocol: UDP (17)
    Header checksum: 0xa1c6 [validation disabled]
    [Header checksum status: Unverified]
    Source: 172.31.72.133
    Destination: 95.250.253.103
  ▼ User Datagram Protocol, Src Port: 4672, Dst Port: 4672
    Source Port: 4672
    Destination Port: 4672
    Length: 200
    Checksum: 0xe800 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 1]
  ▼ Data (192 bytes)
    Data: ac43cc9691b494e490f29b6b30b0b3cda4676aa6793c1436..
    [Length: 192]
```

0000	70 ba ef 9d 95 34 98 83	89 3d 07 18 08 00 45 00	p...
0010	00 dc 46 44 40 00 40 11	a1 c6 ac 1f 48 85 5f fa	..FD
0020	fd 67 12 40 12 40 00 c8	e8 00 ac 43 cc 96 91 b4	..g@
0030	94 e4 90 f2 9b 6b 30 b0	b3 cd a4 67 6a a6 79 3c	
0040	14 36 c6 27 28 c6 53 ed	93 20 a4 56 f9 20 1b 99	..6.
0050	96 e9 8c 19 5e 91 ed e5	ae 33 38 1e 11 28 12 87	
0060	00 12 4a ee 05 0c a2 d0	19 20 bd 33 43 37 e2 b8	..J.
0070	b3 99 89 99 84 f2 67 0b	0a 34 8e 36 b2 be be a9	
0080	6b 70 cf cd 2b db 33 54	a7 4c fc a6 8d 1e f5 aa	kp..
0090	98 fc 03 e6 b1 da ed ea	65 93 96 59 35 9f 8a 86	

```
In [4]: hex(200)
Out[4]: '0xc8'
```

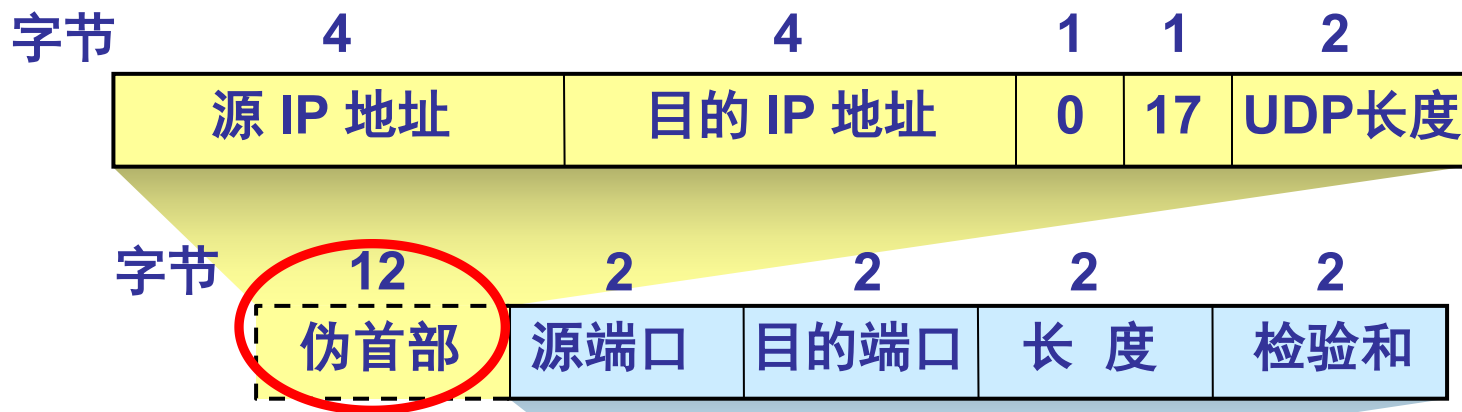
```
In [5]: hex(4672)
Out[5]: '0x1240'
```

```
In [6]:
```

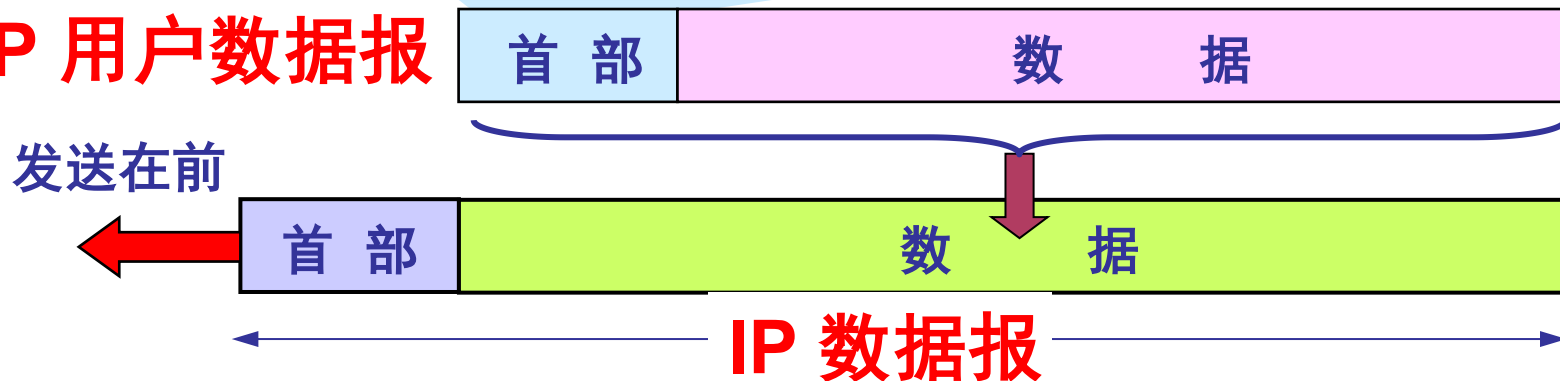


3.3 用户数据报协议UDP

3.3.2 UDP数据报格式



UDP 用户数据报



3.3 用户数据报协议UDP

3.3.3 UDP校验和计算

- 校验和计算：伪报头+UDP数据报
- 伪报头：验证UDP数据报是否正确传送到目的进程

153.18.8.105				10011001 00010010	→	153.18
				00001000 01101001	→	8.105
171.2.14.10				10101011 00000010	→	171.2
				00001110 00001010	→	14.10
0	17	15		00000000 00010001	→	0,17
				00000000 00001111	→	15
1087		13		00000100 00111111	→	1087
				00000000 00001101	→	13
15		0		00000000 00001111	→	15
				00000000 00000000	→	0 (校验和)
T	E	S	T	01010100 01000101	→	T,E
				01010011 01010100	→	S,T
I	N	G	0	01001001 01001110	→	I,N
				01000111 00000000	→	G,0 (填充)



用来计算“校验和”字段的内容。初始设置“校验和”字段=0，然后运行checksum算法对蓝色框内容计算出结果并填入“校验和”字段然后将红色框起的内容发送到网络层

等于UDP总长度，不含伪报头

这是发送到网络层的内容

3.3 用户数据报协议UDP

- 3.3.3 UDP校验和计算
- 注意
 - 求和时遇到的任何溢出都被回卷
- 例子: checksum - two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1